

Cornell University®

CS 4/5782: INTRODUCTION TO DEEP LEARNING

FINAL SUMMARY REPORT

**Reproduction of: A Time Series is Worth 64
Words: Long-term Forecasting with
Transformers**

Eugene Choi (ec727),
Maxwell Pang (mp774),
Elizabeth Tang (et362)

1 Introduction

Long-term forecasting of multivariate time series is essential in applications such as energy management, traffic optimization, and financial planning, yet Transformer-based approaches often face a difficult trade-off between prediction accuracy and computational efficiency. In this project, we aim to reproduce the paper "A Time Series Is Worth 64 Words: Long-Term Forecasting with Transformers" by Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam [2].

In this paper, the authors introduce PatchTST, a vanilla Transformer encoder with two key features. With *patching* the model is fed contiguous subseries rather than individual time steps, and with *channel independence* the single transformer encoder is applied across each univariate series. By combining these strategies, PatchTST not only reduces attention costs but also learns effectively from much longer windows, offering both improved accuracy and efficiency.

2 Chosen Result

For our reproduction, we focused on the multivariate long-term forecasting results with supervised PatchTST as shown in **Table 3** of the original paper. This compares PatchTST/42 against strong time-series Transformer baselines such as the Autoformer, Informer, and Pyraformer with forecast horizons $T = \{96, 192, 336\}$. PatchTST/42 uses 42 input patches, a look-back window of 336 time steps, and patch length $P = 16$ with stride $S = 8$. Additionally, we looked reproduce **Table 4**: multivariate long-term forecasting results with self-supervised PatchTST, which generally show very slightly improved results of the supervised PatchTST. For both of these experiments we used the Electricity Transformer datasets (ETTh1 and ETTm1) [4] to compare across baselines. Both of these tables demonstrate how the PatchTST architecture is able to yield better accuracy than other time-series transformer architecture over most datasets and forecast horizons.

Additionally, we attempted to reproduce the transfer learning results in **Table 5**, where the PatchTST model is first trained on the Electricity dataset [1] then transferred to other datasets. While the original paper transferred the model to the Weather and Traffic datasets, we chose the ETTh1 dataset to transfer to. These results show how pre-training on one dataset and transferring to another dataset yields better results than other models such as the Autoformer and Informer. Because of a mismatch in datasets and parameters, we will compare our transfer learning results to our supervised model results, but we will not compare our transfer learning results to those in **Table 5**.

3 Methodology

In order to implement the PatchTST models described in the paper, we first need to go over the key ideas that differentiate this paper from previous Transformer-based approaches. The namesake of the model, *patching*, describes the process by which we separate the time series into patches of data, which we then feed through a learnable linear embedding to get an encoding of the data. The motivation behind this technique is that now we are able to encode larger amounts of data and longer time horizons in our transformer with the same number of parameters in the transformer itself. Furthermore, this also causes the model to learn representations of short windows rather than individual points, allowing the model to understand local temporal patterns better. Another technique that we used was *RevIN*, which is an acronym for Reversible Instance Normalization. The idea behind *RevIN* is that we want to be able to use the same encoder for all types of data, and so we need to normalize our data before passing it through the transformer.

The paper described two different training methodologies: supervised and self-supervised, as we

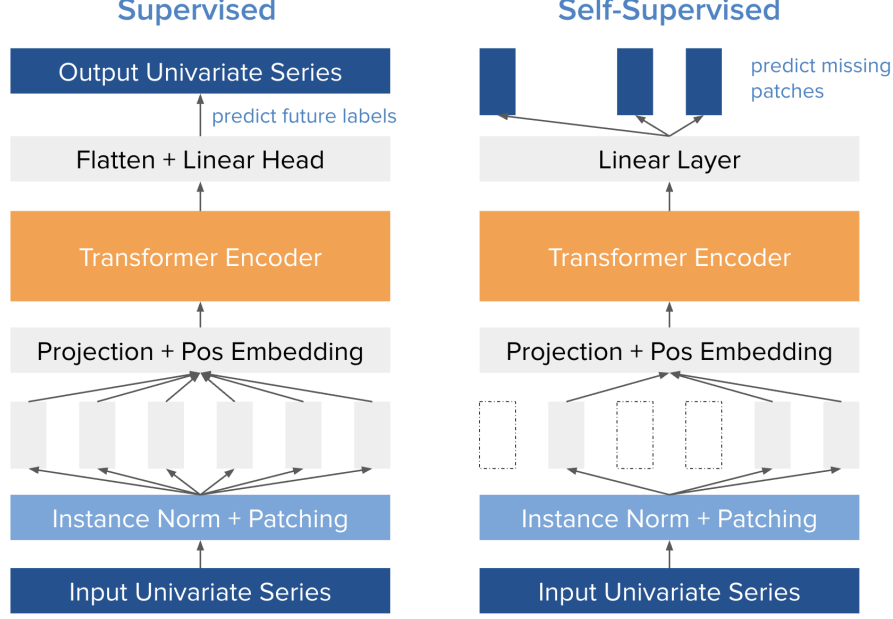


Figure 1: PatchTST architecture. In the supervised learning setup, the model is trained to predict the next horizon of data. In the self-supervised learning setup, a subset of patches is randomly masked (set to zero), and the model learns to reconstruct the missing patches.

can see in the figure below. In the supervised approach, we take the information from the past and attempt to predict the future. On the other hand, in the self-supervised approach, we mask some of the patches from the past and try to predict them using the surrounding patches. Although the two methods are very different, they share the same transformer encoder architecture and patch encoding linear layer, allowing us to pre-train using self-supervised training and fine-tune with supervised training. This is especially useful in small datasets where we lack many data points and require human supervision to create more labeled data.

Furthermore, another useful process described in the paper is transfer learning. Essentially, we claim that we can transfer the patch embeddings and transformer encoder weights from one dataset to another, and it will speed up the training process significantly, even if the datasets describe completely independent processes and have different numbers of variables. We suspect that this is largely due to the patches capturing local temporal patterns, which the encoder then uses to learn long-range dependencies.

4 Results & Analysis

We re-implemented the PatchTST model for long-term time series forecasting and evaluated it on the ETTh1 and ETTm1 datasets across three forecast horizons: 96, 192, and 336. While our results followed the same general performance trends as reported in the original paper, our MSE scores (refer to table 1) and MAE scores (not shown) were consistently higher across all settings. We attribute this discrepancy primarily to computational limitations, which constrained us to fewer training epochs (about one-tenth of the original) and prevented us from running on the full set of datasets or including the 720-step horizon. Additionally, while the original authors mention using a vanilla Transformer, their implementation reportedly included BatchNorm layers and other design choices that likely contributed to their superior performance. Despite these limitations, our results still reflected the paper’s core insight: PatchTST performs comparably or better than other architectures for time series Transformers.

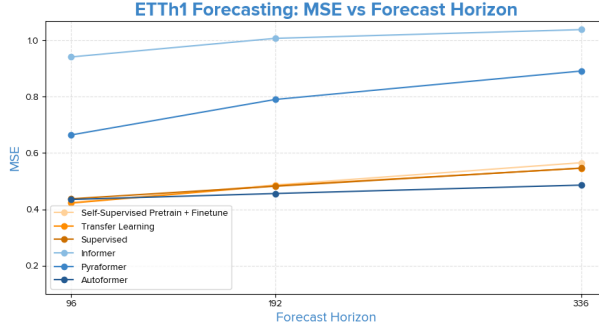


Figure 2: Comparison of our PatchTST models vs Previous Transformer-based Models

In our transfer learning experiments using the Electricity dataset, we found that fine-tuning a self-supervised pretrained model yielded similar performance to fully supervised training, even with half the number of epochs, supporting the claim that transfer learning is both effective and efficient in this context. Interestingly, we did not observe meaningful differences between models trained with supervised versus fine-tuned self-supervised objectives which differs from what was stated in the paper. However, we can possibly attribute this to the architectural and computational differences we discussed earlier. We also found that shorter forecast horizons consistently led to lower error, and hyperparameter variations such as patch size, stride, and lookback window had minimal impact on final loss which was similar to what they discovered in the paper.

Dataset Method	Forecast Horizon	Our PatchTST MSE		Original PatchTST MSE Supervised
		Supervised	Transfer	
ETTh1	96	0.437	0.423	0.375
	192	0.481	0.507	0.414
	336	0.546	0.557	0.431
ETTm1	96	0.335	0.335	0.290
	192	0.372	0.381	0.332
	336	0.417	0.412	0.366

Table 1: Comparison of MSE for Our PatchTST supervised and transfer learning models vs. Original PatchTST across different datasets and forecast horizons.

5 Reflections

Through our re-implementation of PatchTST, we found that patching and channel-wise independence led to better performance and faster training compared to other Transformer-based models. We saw minimal differences between supervised training and fine-tuned self-supervised learning, and our transfer learning experiments confirmed that pretrained models can achieve similar results with fewer epochs when compared to end-to-end fine-tuning. While our loss values were slightly higher than those reported in the original paper, likely due to differences in the encoder, we observed expected trends such as lower loss at shorter horizons and no significant performance change from varying patch size, stride, or lookback window. In future work, we plan to explore learnable positional encodings, integrate BatchNorm layers, and apply PatchTST to real-world datasets in finance, healthcare, and environmental monitoring.

References

- [1] Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term temporal patterns with deep neural networks, 2018.
- [2] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers, 2023.
- [3] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting?, 2022.
- [4] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *The Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Virtual Conference*, volume 35, pages 11106–11115. AAAI Press, 2021.